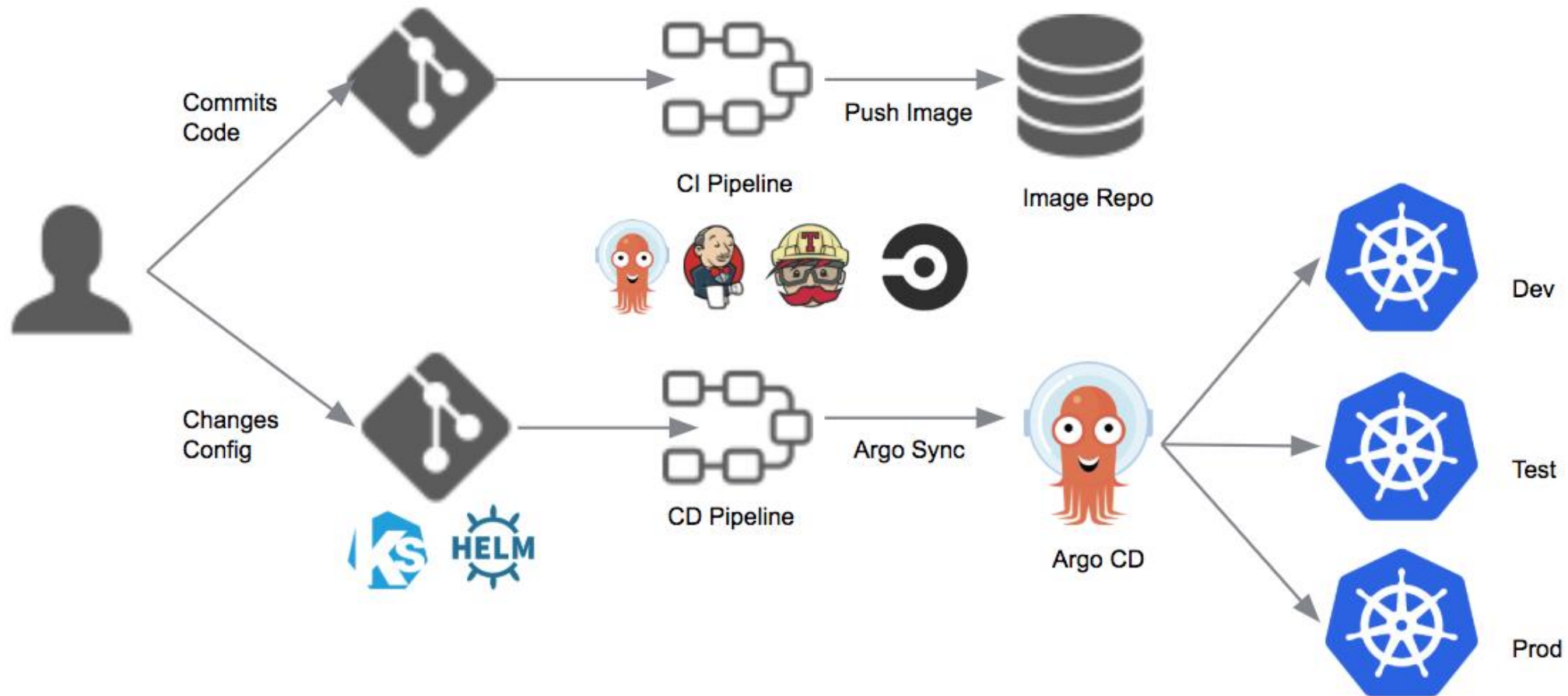


Developer Operations (devops)

Adaptions to Deployment



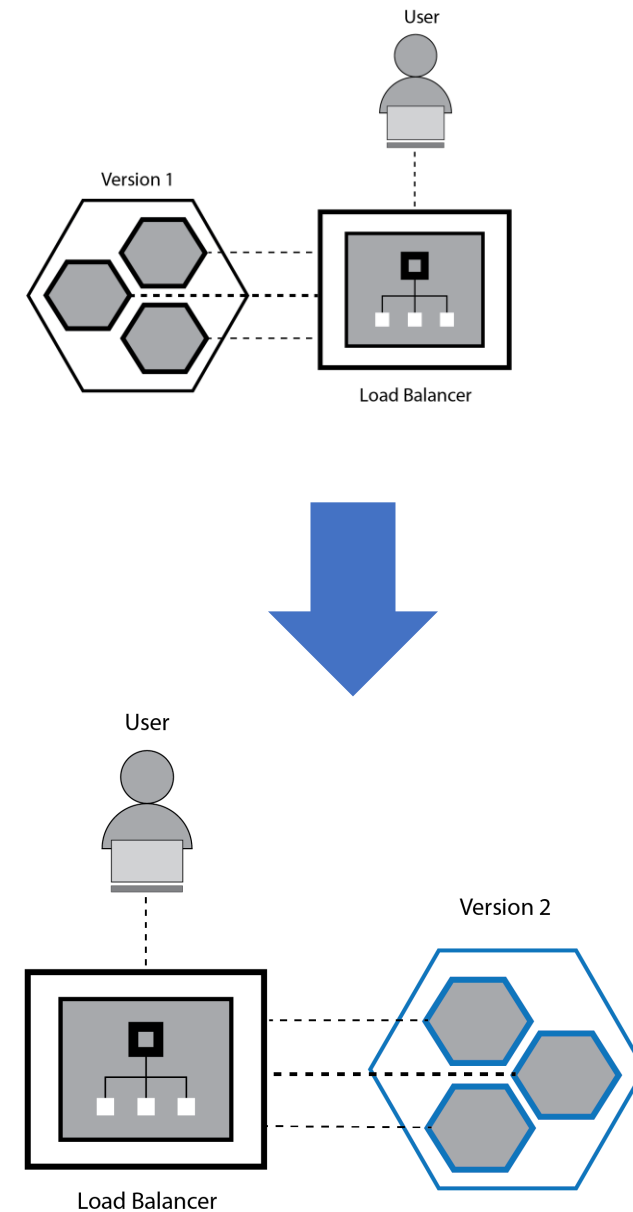
Recap: How to deploy?



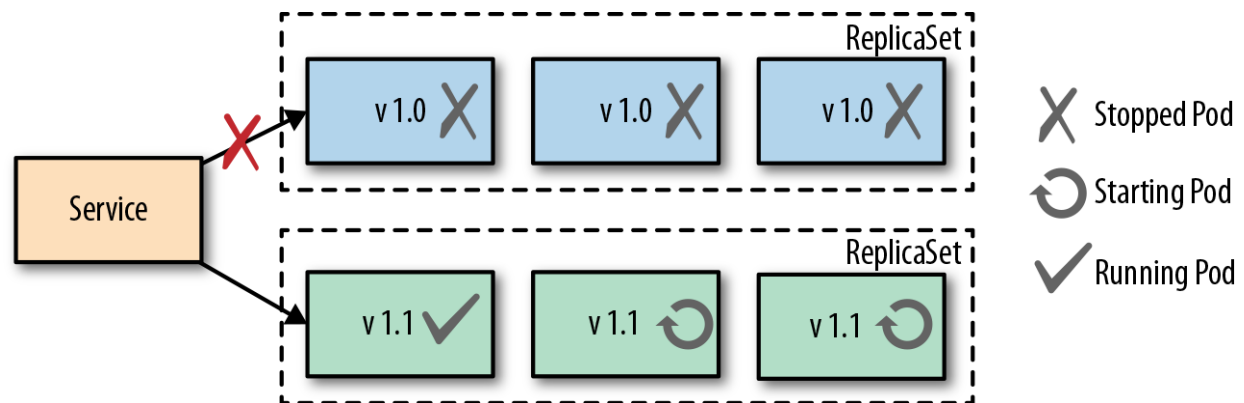
How do you rollout?

Recreate Deployment

Removes old versioned pods by replacing them: all at one time



Recreate Deployment, Implementation



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: random-generator
spec:
  replicas: 3
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: random-generator
  template:
    metadata:
      labels:
        app: random-generator
    spec:
      containers:
        - image: k8spatterns/random-generator:1.0
          name: random-generator
          readinessProbe:
            exec:
              command: [ "stat", "/random-generator-ready" ]
```

Recreate Deployment

Removes old versioned pods one by one by replacing it.

Pros:

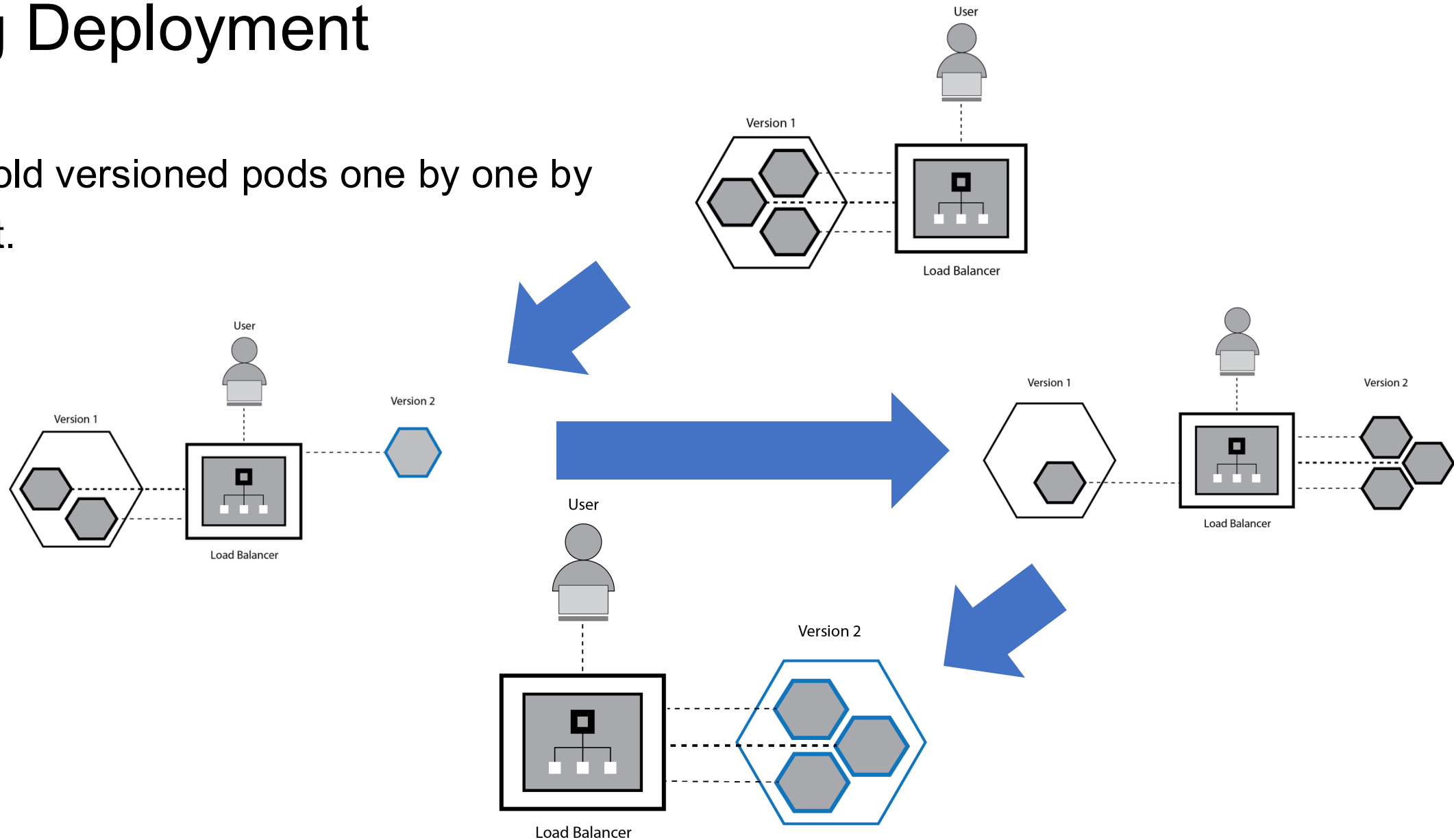
- Einfach
- Nur eine Version erreichbar, Breaking Changes
- Steuerung exakter Zeitpunkt
- ApplikationState, auch extern komplett deployt

Cons:

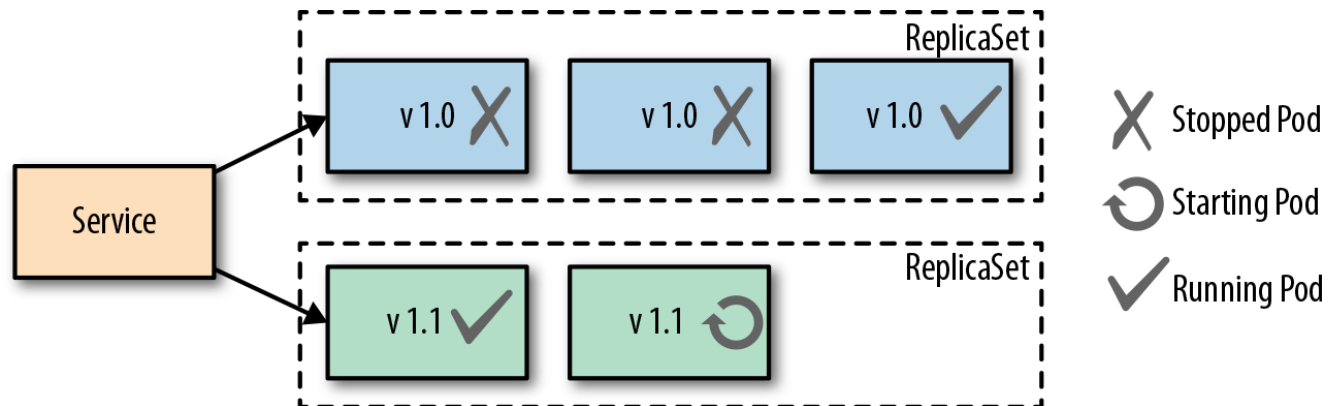
- Downtime

Rolling Deployment

Removes old versioned pods one by one by replacing it.



Rolling Deployment, Implementation



```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: random-generator
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: random-generator
  template:
    metadata:
      labels:
        app: random-generator
    spec:
      containers:
        - image: k8spatterns/random-generator:1.0
          name: random-generator
          readinessProbe:
            exec:
              command: [ "stat", "/random-generator-ready" ]

```


Rolling Deployment

Removes old versioned pods one by one by replacing it.

Pros:

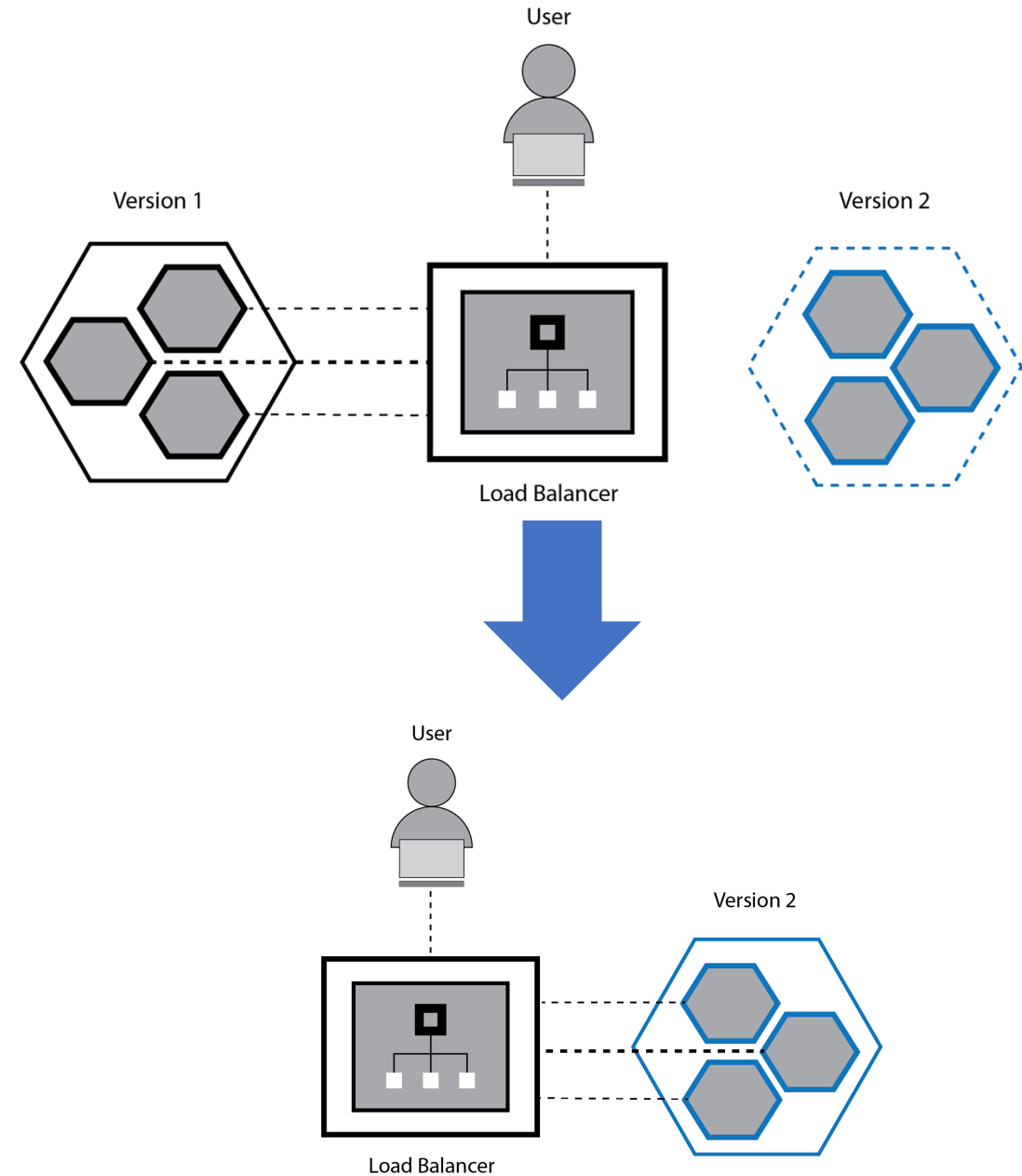
- Keine Downtime
- Rollback bei der Applikation
- Easy to implement

Cons:

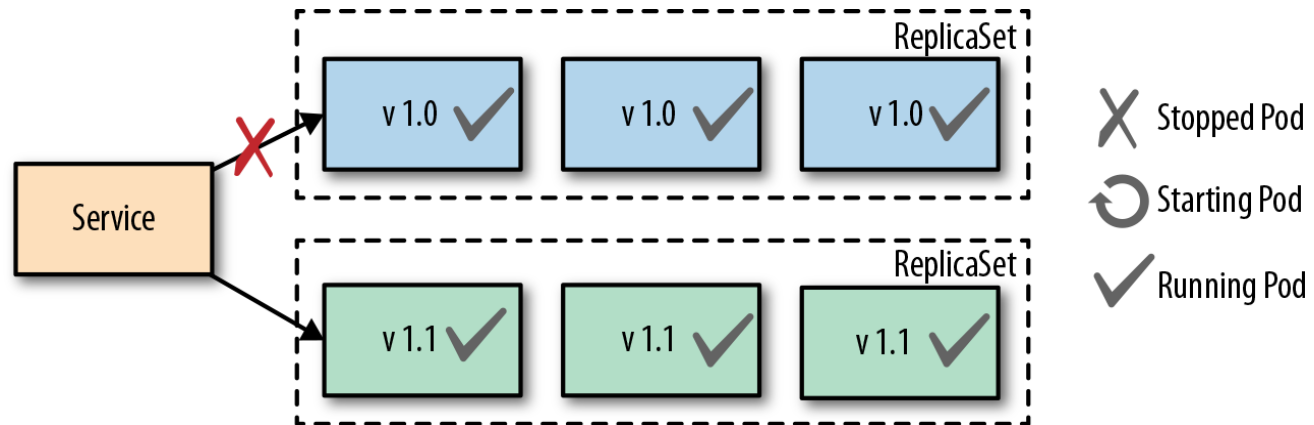
- Seiteneffekt / Umsysteme
- “Unkontrolliert”, Zeit wie auch Traffic

Blue/Green Deployment

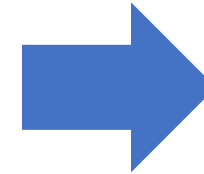
Duplicate deployment, new version is deployed aside to old one. Switching traffic to new version.



Blue/Green Deployment



```
apiVersion: v1
kind: Service
metadata:
  name: service
spec:
  selector:
    app: myapp
    env: blue
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```



```
apiVersion: v1
kind: Service
metadata:
  name: service
spec:
  selector:
    app: myapp
    env: green
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Blue/Green Deployment

Duplicate deployment, new version is deployed aside to old one. Switching traffic to new version.

Pros:

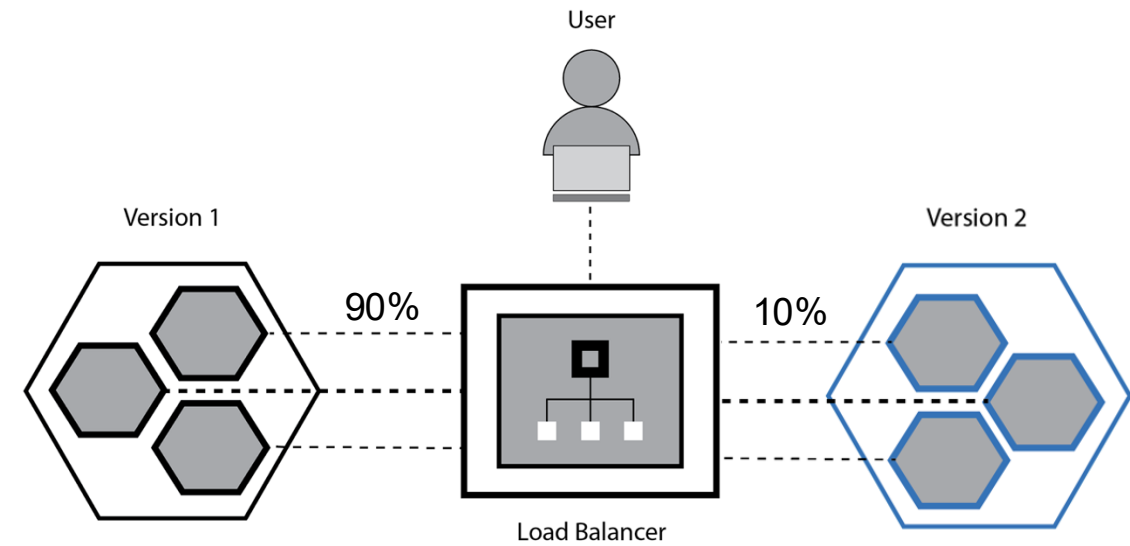
- Zeitliche Kontrolle, atomar
- Rollbackfähigkeit (beware Umsysteme)
- No Downtime

Cons:

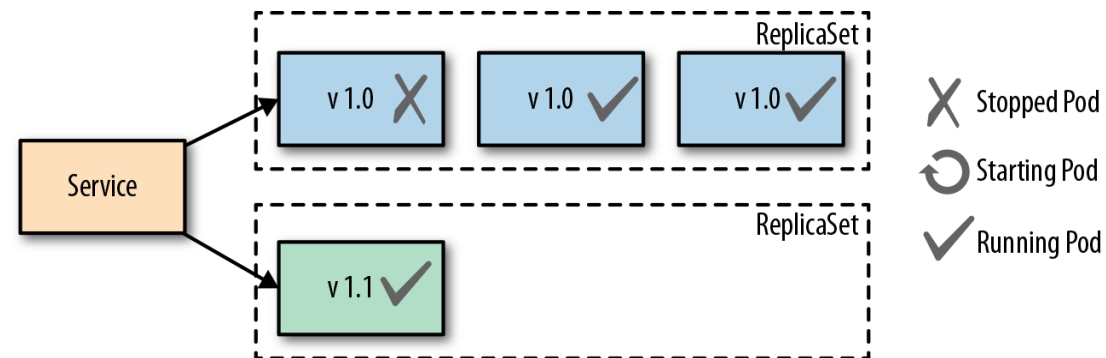
- Doppelte Ressource

Canary Deployment

Redirecting small portions of traffic to new release.



Canary Deployment, Implementation



```
apiVersion: v1
kind: Service
metadata:
  name: service
spec:
  selector:
    app: random-generator
ports:
  - protocol: TCP
    port: 80
    targetPort: 80
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: random-generator-1
spec:
  replicas: 3
  selector:
    matchLabels:
      app: random-generator
  template:
    metadata:
      labels:
        app: random-generator
    spec:
      containers:
        - image: k8spatterns/random-generator:1.0
          name: random-generator
          readinessProbe:
            exec:
              command: [ "stat", "/random-generator-ready" ]
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: random-generator-2
spec:
  replicas: 1
  selector:
    matchLabels:
      app: random-generator
  template:
    metadata:
      labels:
        app: random-generator
    spec:
      containers:
        - image: k8spatterns/random-generator:2.0
          name: random-generator
          readinessProbe:
            exec:
              command: [ "stat", "/random-generator-ready" ]
```

Canary Deployment

Redirecting small portions of traffic to new release.

Pros:

- Impact auf Userbasis wird minimiert
- Kontrolle / zeitliche Planbarkeit
- Keine signifikanten Mehrkosten

Cons:

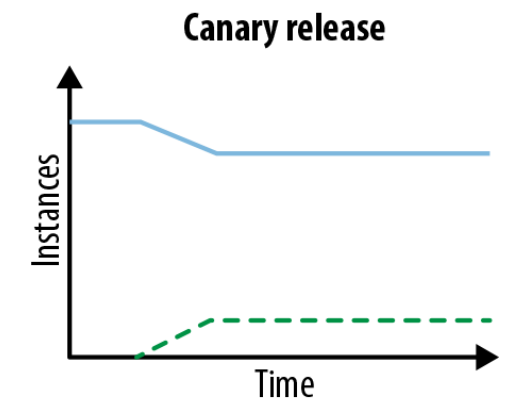
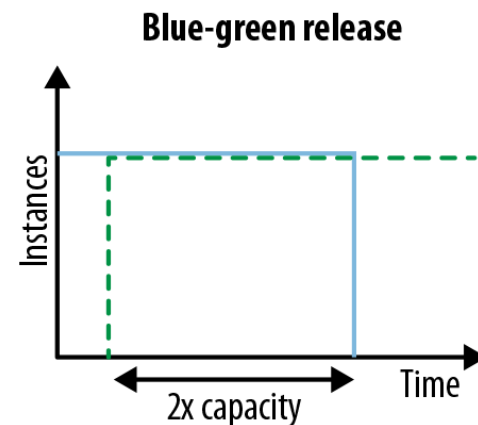
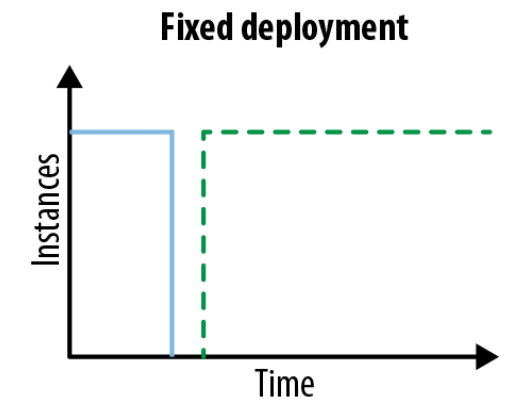
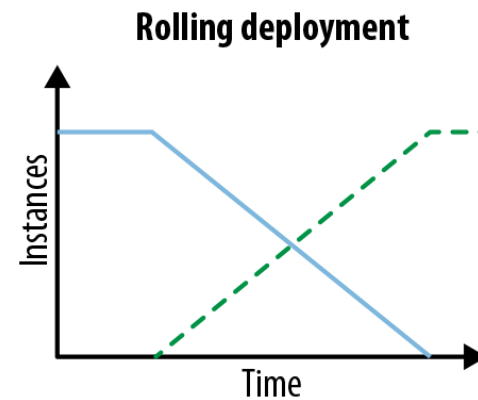
- Multi Api Support
- Complicated to implement, other tools become necessary such as service mesh

Comparison, Deployment Patterns

Different Patterns for different use cases:

- Utilized external systems like databases?
- Costs for duplicate infrastructure?
- Rollback ability regarding the application?
- Additional traffic redirection techniques like service mesh available?

There is no one-size-fits-all...



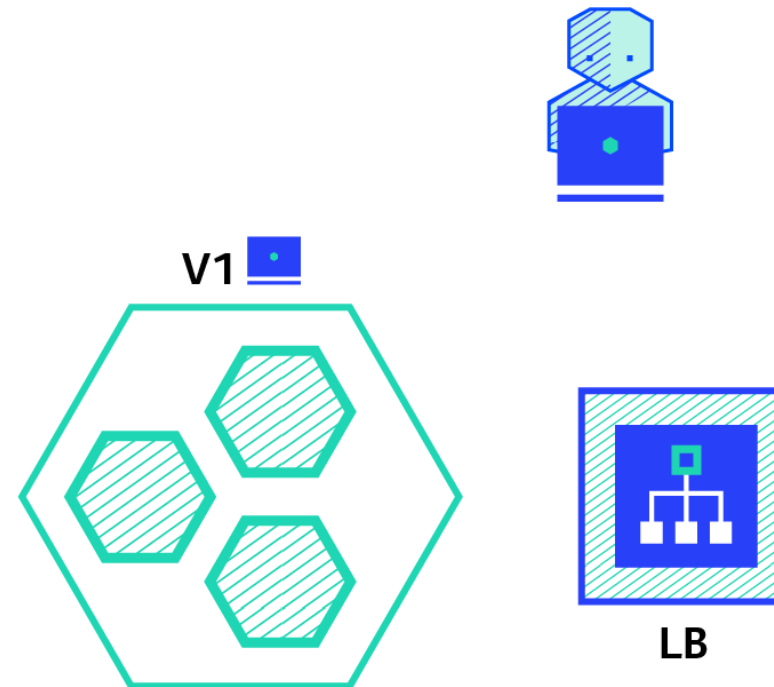
Additional Deployment Strategies, A/B Testing

Traffic for a given part of users is redirected

+ Control about traffic

+ Opt-in from user side

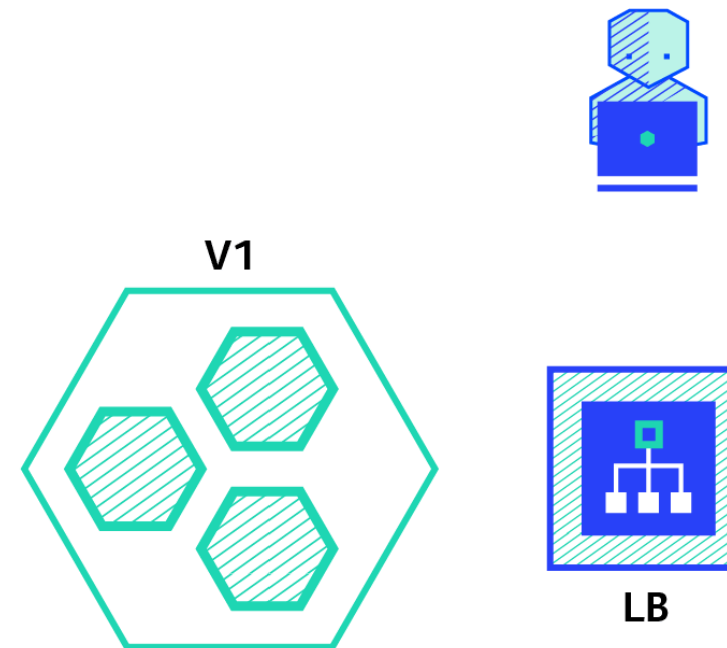
- Complex setup, semantic load balancing
- Complex to trace errors, especially in backing services



Deployment Strategies, Shadowing

Requests are sent to new release as well,
acting as a shadow

- + no direct user impact
- + load testing without load testing
- + switch when everything is ready
- Double infrastructure
- Entire shadowing can become complex with respect to backing services
- Complex to set up



Deployment Strategies, Summary

Strategy	ZERO DOWNTIME	REAL TRAFFIC TESTING	TARGETED USERS	CLOUD COST	ROLLBACK DURATION	NEGATIVE IMPACT ON USER	COMPLEXITY OF SETUP
RECREATE version A is terminated then version B is rolled out	✗	✗	✗	■ □ □	■ ■ ■	■ ■ ■	□ □ □
RAMPED version B is slowly rolled out and replacing version A	✓	✗	✗	■ □ □	■ ■ ■	■ □ □	■ □ □
BLUE/GREEN version B is released alongside version A, then the traffic is switched to version B	✓	✗	✗	■ ■ ■	□ □ □	■ ■ □	■ ■ □
CANARY version B is released to a subset of users, then proceed to a full rollout	✓	✓	✗	■ □ □	■ □ □	■ □ □	■ ■ □
A/B TESTING version B is released to a subset of users under specific condition	✓	✓	✓	■ □ □	■ □ □	■ □ □	■ ■ ■
SHADOW version B receives real world traffic alongside version A and doesn't impact the response	✓	✓	✗	■ ■ ■	□ □ □	□ □ □	■ ■ ■

Literature

